



Pontifícia Universidade Católica do Rio Grande do Sul  
Confiabilidade e Segurança de Software  
98G08-04

## Trabalho Prático: Logaritmo Discreto

Prof. Iaçanã Ianiski Weber

Entrega: 22/06 (23:59) - no Moodle

### Contexto: Diffie-Hellman e o Problema do Logaritmo Discreto

O protocolo de troca de chaves de **Diffie-Hellman** permite que duas partes estabeleçam um segredo compartilhado sobre um canal público. Alice e Bob acordam em um primo  $p$  e um gerador  $\alpha \in (\mathbb{Z}/p\mathbb{Z})^*$ ; cada um sorteia um expoente secreto ( $a$  e  $b$ ), publica  $A = \alpha^a \bmod p$  e  $B = \alpha^b \bmod p$ , e calcula  $K_{ab} = B^a \bmod p = A^b \bmod p = \alpha^{ab} \bmod p$ .

A segurança desse esquema repousa no **Problema do Logaritmo Discreto (DLP)**: dados  $\alpha$ ,  $p$  e  $\alpha^x \bmod p$ , recuperar  $x$  é computacionalmente difícil quando os parâmetros são bem escolhidos. Neste trabalho você atacará o DLP em  $(\mathbb{Z}/p\mathbb{Z})^*$  para uma sequência crescente de tamanhos de  $p$ .

## O Problema

Para cada cenário  $C_i$  é fornecida uma quádrupla

$$(\alpha, p, A, B), \quad A = \alpha^a \bmod p, \quad B = \alpha^b \bmod p,$$

e o seu objetivo é recuperar a chave compartilhada de Diffie-Hellman

$$K_{ab} = \alpha^{a \cdot b} \bmod p.$$

Você **não** precisa reportar  $a$  nem  $b$ , apenas  $K_{ab}$  em decimal. A correção compara cada  $K_{ab}$  entregue, byte a byte, com o gabarito do professor.

## Estrutura dos arquivos

- **desafios.txt**: entrada do trabalho. Contém  $(\alpha, p, A, B)$  para cada cenário. Texto puro, decimal, fácil de parsear com `split` em qualquer linguagem.
- **solucao.txt**: modelo a preencher e entregar. Preencha  $K_{ab}$  em cada cenário resolvido; deixe em branco (ou comentado com `#`) os que você não conseguir atacar.

## Cenários

São **23 cenários** de Diffie-Hellman, com  $p$  variando de 16 até 1024 bits. O cenário  $C_0$  (16 bits) acompanha a solução completa; use-o para validar sua implementação antes de partir para os demais.

| Cenário         | bits( $p$ ) | Cenário  | bits( $p$ ) | Cenário  | bits( $p$ ) |
|-----------------|-------------|----------|-------------|----------|-------------|
| $C_0$ (exemplo) | 16          | $C_8$    | 64          | $C_{16}$ | 128         |
| $C_1$           | 32          | $C_9$    | 80          | $C_{17}$ | 256         |
| $C_2$           | 32          | $C_{10}$ | 80          | $C_{18}$ | 256         |
| $C_3$           | 40          | $C_{11}$ | 96          | $C_{19}$ | 512         |
| $C_4$           | 40          | $C_{12}$ | 96          | $C_{20}$ | 512         |
| $C_5$           | 48          | $C_{13}$ | 112         | $C_{21}$ | 1024        |
| $C_6$           | 48          | $C_{14}$ | 112         | $C_{22}$ | 1024        |
| $C_7$           | 64          | $C_{15}$ | 128         |          |             |

### Exemplo resolvido: $C_0$ (16 bits)

O cenário  $C_0$  é fornecido com  $a$ ,  $b$  e  $K_{ab}$  revelados em `desafios.txt`. Use esse cenário como *sanity check*: implemente seu ataque, rode contra  $C_0$ , e confirme que o  $K_{ab}$  recuperado coincide com o publicado antes de avançar para os cenários reais. Se a implementação falha em 16 bits, não fará sentido tentar 128.

## Tarefas

1. **Pesquise** os principais ataques ao DLP em  $(\mathbb{Z}/p\mathbb{Z})^*$  e sintetize, em poucas linhas, as ideias centrais e a complexidade assintótica de cada um:
  - a) Força bruta ( $\mathcal{O}(p)$ ),
  - b) Baby-step Giant-step (BSGS,  $\mathcal{O}(\sqrt{q})$  tempo e memória, onde  $q = \text{ord}(\alpha)$ ),
  - c) Pollard rho ( $\mathcal{O}(\sqrt{q})$  tempo,  $\mathcal{O}(1)$  memória),
  - d) Pohlig–Hellman (reduz ao DLP nos subgrupos de ordem prima de  $q$ ),
  - e) Index Calculus / Number Field Sieve (subexponencial em  $p$ ).
2. **Implemente** um ou mais ataques na linguagem que preferir. Bibliotecas de aritmética arbitrária são encorajadas: `int` nativo do Python, `gmpy2`, `BigInteger` em Java, `mpz` (GMP) em C/C++.
3. **Execute** seu ataque contra  $C_0$  a  $C_{22}$ , registrando o tempo até obter  $K_{ab}$  ou desistir. O timeout sugerido é de **1 hora por cenário**, mas a escolha é sua. Resolva todos os que conseguir; para os demais, projete o tempo estimado.
4. **Gráfico** tempo  $\times$  bits( $p$ ) em escala log no eixo  $y$ . Inclua:
  - os pontos medidos para os cenários efetivamente resolvidos,
  - uma linha de **projeção** para os cenários não alcançados, **justificada** pela complexidade assintótica da técnica utilizada.
5. **Reporte a infraestrutura computacional** usada. Este item é *obrigatório*, pois sem ele o tempo medido é incomparável e a projeção perde valor:
  - modelo da CPU (e/ou GPU), número de núcleos e *threads* efetivamente usados;
  - memória RAM disponível;
  - sistema operacional;
  - se o ataque for distribuído: **topologia** (quantas máquinas, como foram coordenadas, latência da rede, divisão de trabalho).
6. **Relatório** de no máximo **2 páginas** (não conta capa nem anexos) contendo: técnica utilizada, gráfico, tabela de tempos, descrição da infraestrutura e **discussão crítica** sobre o que os cenários revelam sobre escolha de parâmetros em Diffie–Hellman real.

### Entregáveis

- **relatorio.pdf**: máx. 2 páginas, contendo técnica, gráfico tempo  $\times$  bits( $p$ ) com projeção, tabela de tempos, descrição da **infraestrutura computacional** (obrigatório) e discussão.
- **solucao.txt**: preenchido com  $K_{ab}$  por cenário resolvido, no formato fornecido.
- **Código**: *não* incluir no corpo do relatório. Se imprescindível, anexar como apêndice ou disponibilizar em **repositório público** (GitHub, GitLab). Não conta para o limite de 2 páginas.